CSE221: Algorithms

Assignment-1, Summer 26

Time complexity, Searching, Sorting

Submission Deadline:9/7/26 by 5.00 pm

You Should include a Top Page with you submission containing your Name, ID, Section and Semester. Answers should be handwritten.

---

1. Solve the following questions about time complexity.
   a. Calculate the time complexity of the following code snippet:

```
int p = 0;
for (i = n / 2; i > 1; i /= 6) {
      for (j = 2; j <= n; j *= 4) {
            for (k = i; k <= j; k *= 3) {
                  p = p + n / 2;
            }
      }
}
```

   b. Calculate the time complexity of the following code snippet:

```
int p = 0;
for (i = n; i > 1; i -= 1) {
      for (j = 2; j <= n; j += 1) {
            p = p + n;
            if (p > (n ^ i)) // ^ sign means XOR in code
            {
                  break;
            }
      }
}
for (i = n / 2; i > 1; i /= 6) {
      for (j = 2; j <= i; j *= 4) {
            for (k = 0; k <= j; k *= 3) {
                  p = p + n / 2;
            }
      }
}
```

2. After working with the binary search algorithm, as a CSE221 student you want to explore its strength. You have several ideas in your mind. Try to modify the algorithm to implement your ideas.
   a. What if you want to return the first index of the search key in case there are duplicates. For example: your search key is 13 and there are three 13s in the list.
   b. Now, you also want to return the number of times the key appears in the list. Say, the search key is 54 and it occurs 4 times in indices 6,7,8,9. Then you should return (6,4).

3.

Consider the following list:

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

Will the algorithm given on the right be able to find the search value T=2 for the list above?
If yes, **show** the value of L, R, m in each step of the algorithm.
If no, **explain** why not.

```
function binary_search(A, n, T) is
    L := 0
    R := n - 1
    while L ≤ R do
        m := floor((L + R) / 2)
        if A[m] < T then
            L := m + 1
        else if A[m] > T then
            R := m - 1
        else:
            return m
    return unsuccessful
```

Here, ***A*** denotes the list, and ***n*** denotes its size.

4. You are given an array containing N distinct integers in a wave-like sequence. Meaning, the numbers in the beginning are in ascending order, and after a specific position, they are in descending order. For example: [1, 3, 4, 5, 9, 6, 2, -1]
You have to find the maximum number of this sequence. Can you devise an efficient algorithm such that the time complexity will be less than O(N)?

- **i.** Present your solution idea as a pseudocode/ python code/ flowchart/ step-by-step instructions/ logical explanation in one-two paragraphs.
- **ii.** Write the time complexity of your algorithm.

5. Your friend gave you an integer (key) and asked you to find its integer square root without using built-in functions like sqrt() or exponentiation. For example, if the input is 25, the output should be 5 because 5^2=25. If the square root is not an integer, return the floor value of the actual square root. For example, for an input of 10, the output should be 3.
Your friend's initial approach is to use linear search (time complexity O(n)) as shown below.

```
def LinearSearchToFindSquareRoot(key):
    result = -1
    for i in range(1, key, 1):
        if i * i <= key:
            result = i
    return result
```

a. Present an $O(\sqrt{n})$ time algorithm with pseudocode/programmable code/step-by-step logical instructions.
b. Present an $O(log_2 N)$ time algorithm with pseudocode/programmable code/step-by-step logical instructions.

6. A sorted array contains pairs of identical elements except one element. Generate an algorithm with pseudocode/programmable code/step-by-step logical instructions to find the unique element.

For Example: 2 2 5 5 7 8 8 9 9, find the unique element.

CSE221 Assignment 2[WBH]

Summer 2026

Section 09

Instructions:

- Assignment must be handwritten
- Naming convention: **StudentID_A2**, example: 12101100_A2
- Submit scanned pdf in this google form:
- https://forms.gle/oqnZps1Hvv9MMdGm7
- Deadline(no extensions): **21st August, 2026 (11:59Pm)**

Ensure assignment is submitted only using this google form, submission elsewhere or after the deadline will not be accepted. Must submit using your g-suite email.

1.

Dhaka, especially your area, is facing an Electricity crisis. Many initiatives including load shedding, reducing office hours, etc have already been taken. The local authority of your area is looking for feasible solutions to reduce the consumption of electricity. One of the members of their advisory committee has suggested that they should reduce the operating costs of road lighting. Till now every road is illuminated all night long. To reduce electricity consumption, they have decided to no longer illuminate every road, but to switch off the road lighting of some roads. To make sure that the inhabitants of your area still feel safe, they want to optimize the lighting in such a way that after darkening some roads at night, there will still be at least one illuminated path from every major point in your area to every other major point.

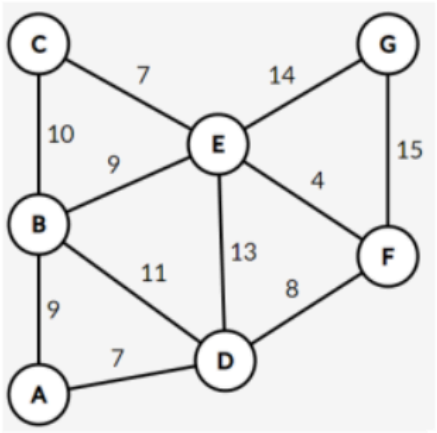


Figure : Graph of your area where the Vertices denote major points and Edges denote electricity consumption in a day.

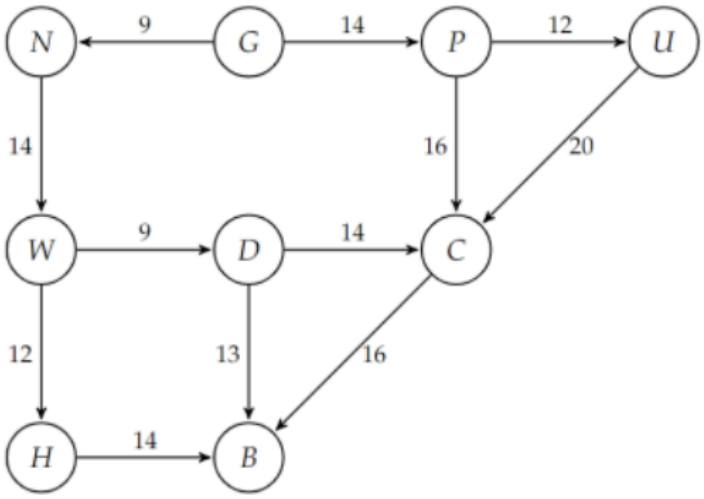
a. CO3 **Apply** a suitable Algorithm to help the authority determine the roads they need to illuminate and the cost which will minimize the electricity consumption. If you need to consider any root vertex for your Algorithm, you can consider 'A' as such.

b. CO5 Observe the Graph of your area and your provided solution of question 1(a) and **determine** the maximum electricity of a day the authority can save.

c. CO6 The United Arab Emirates (UAE), especially Dubai, is hosting the Asia Cup 2022. They too want to minimize the electricity consumption by switching off the lights in some roads leaving exactly one illuminated path from every major point in your area to every other major point. However, to keep the tourists entertained, the Authority of Dubai has taken an opposite approach to yours. They want to illuminate the roads which require the most electricity.
Propose an algorithm to compute the path which fulfills this requirement.

2.

Consider the network of roads connecting a set of cities given by the graph below.



a) **Simulate** Dijkstra's algorithm to find the weight of the shortest path from G to B.

b) There is a proposal to add one new road to this network, and there is a list of five pairs of cities between which the new road can be built. Each such potential road has an associated length. As a designer for the public works department, you are asked to determine the road whose addition to the existing network would result in the maximum decrease in the distance from G to B. One way to do this would be to add each of the five edges separately, run Dijkstra's algorithm five different times, and then compare the results. But can you do this by invoking Dijkstra's algorithm only two times (and possibly doing a linear amount of extra work)?

Present your idea with a pseudo-code/flowchart/step by step instruction/algorithm explanation.

3.

Professor Bumstead needs to figure out a proper order of clothing when getting dressed. He notes down some of the dependencies which describe which garments he needs to put on before others. For example, Prof Bumstead needs to put on his shirt before he puts on his tie or his belt. He wrote down the following:

1. undershorts, watch and socks are independent
2. pants need to be put on after undershorts
3. shoes need to be put on after pants, undershorts and socks
4. belt needs to be put on after pants and shirt
5. tie needs to be put on after shirt
6. jacket needs to be put on after tie and belt

a. **Write** down the name of an appropriate algorithm to find the proper order of clothing items.
b. **Simulate** the algorithm by drawing a proper graph and determining the correct order of clothing items. If you can choose among multiple items, choose the alphabetically smallest one. For example, between “shirt” and “shoes”, i comes before o, so choose “shirt”.

CSE221 Assignment 3[WBH]

Summer 2026

Section 09

Instructions:

- Assignment must be handwritten
- Naming convention: **StudentID_A3**, example: 12101100_A3
- Submit scanned pdf in this google form: https://forms.gle/VzPMPioB4JkG2cNU9
- Deadline(no extensions): **11th September, 2026 (11:59Pm)**
- **Question 1 and 2g are optional**

Ensure assignment is submitted only using this google form, submission elsewhere or after the deadline will not be accepted. Must submit using your g-suite email.

Question 1 [Optional] **(10 marks)**

Place N queens on an N x N chessboard so that no two queens attack each other. For this question, use N = 8 and place one queen in each column.

(a) State the three conflict conditions that must be checked before placing a queen at row r and column c. Explain how rows and the two diagonal directions can be represented for fast safety checks. **[2 marks]**

(b) Write a recursive backtracking pseudocode that tries rows 1 through 8 in each column, places a safe queen, recurses to the next column, and removes the queen when that branch fails. **[4 marks]**

(c) Give one valid arrangement of eight queens as a list where the value at position c is the selected row for column c. Verify briefly that no two listed queens share a row or diagonal. **[2 marks]**

(d) Explain how pruning improves on generating every board arrangement. State a reasonable worst-case time bound and the auxiliary-space complexity of the backtracking solution. **[2 marks]**

Question 2 **(10 marks)**

A lab has one machine and receives the following half-open reservation requests [start, finish).

A	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**
[1,4)	[3,5)	[0,6)	[5,7)	[3,9)	[5,9)	[6,10)	[8,11)	[8,12)	[2,14)	[12,16)

(e) Apply the earliest-finish-time greedy algorithm. Show the sorted order, each accept/reject decision, and the maximum compatible set. **[4 marks]**

(f) Give a brief exchange-argument justification for why choosing the compatible interval that finishes earliest is optimal for the unweighted problem. **[2 marks]**

(g) (optional)

For the weighted requests P=[1,3), value 5; Q=[3,5), value 5; and R=[1,5), value 13, show that earliest-finish-time greedy is not optimal. Explain why weighted interval scheduling needs a different method, and write the dynamic-programming recurrence using p(j), the last request compatible with j. **[4 marks]**

Question 3 (10 marks)

A telemetry system observes the following symbol frequencies. When frequencies tie, place the alphabetically smaller symbol or subtree on the left. Treat left as 0 and right as 1.

A	B	C	D	E	F
32	25	20	12	7	4

(h) Simulate Huffman's greedy algorithm using a min-priority queue. Show every merge and draw the final Huffman tree. **[4 marks]**

(i) Write the codeword for each symbol and calculate the total number of bits needed to encode all 100 symbol occurrences. **[3 marks]**

(j) Encode the message ABACA. Compare its length with fixed-length coding for six symbols and state the number of bits saved or lost. Ignore the code-table overhead. **[2 marks]**

(k) Explain briefly how the greedy choice in Huffman coding leads to an optimal prefix-free code. **[1 marks]**

Question 4 (10 marks)

In the 15-puzzle, the blank may swap with an adjacent tile. Consider the following initial and goal states.

	Initial state					Goal state		
1	2	3	4		1	2	3	4
5	6	7	8		5	6	7	8
9	10	11	12		9	10	11	12
_	13	14	15		13	14	15	_

(aa) Describe a state representation, the legal moves, and the inversion-parity test used to reject an unsolvable 4x4 puzzle before searching. **[2 marks]**

(bb) Write recursive backtracking pseudocode that uses a visited-state set, a path list, a depth limit, pruning, and undoing of moves. The procedure should return a solution path if one is found. **[4 marks]**

(cc) Using move order Right, Up, Left, Down, trace the successful branch from the initial state through the first three recursive levels and write the move sequence that reaches the goal. **[2 marks]**

(dd) Explain why plain depth-first backtracking does not guarantee the shortest solution. Name a method that does guarantee the shortest number of moves and compare its memory use with DFS backtracking. **[2 marks]**

Question 5 (10 marks)

A rescue aircraft has a capacity of 10 weight units. The available supply bundles are listed below.

Bundle	Weight	Value	Value/weight
A	2	60	30
B	4	100	25
C	6	120	20
D	5	95	19

(ee) Explain the difference between fractional and 0-1 knapsack, including whether an item may be divided and the standard algorithmic approach for each. **[2 marks]**

(ff) Solve the fractional-knapsack version using the greedy value-to-weight ratio. Show the selected fractions and maximum value. **[3 marks]**

(gg) For the 0-1 version, write the recurrence and base cases, construct the DP table for capacities 0 through 10, and report the selected bundles and maximum value. **[4 marks]**

(hh) Use your two results to explain why the fractional greedy rule is not guaranteed to solve 0-1 knapsack. **[1 marks]**

Question 6 (10 marks)

Let X = ABCBDAB and Y = BDCABA. For LCS(i, j), use the first i characters of X and the first j characters of Y.

(ii) Write the complete LCS recurrence relation, including base cases, and explain what each DP cell represents. **[3 marks]**

(jj) Construct the full dynamic-programming table, determine the LCS length, and backtrack to produce one valid LCS string. If several answers exist, any one is acceptable. **[4 marks]**

(kk) Explain overlapping subproblems and optimal substructure in the context of LCS. Refer to specific recursive subproblems to show how both properties arise. **[2 marks]**

(ll) State the DP time and space complexity. If only the LCS length is needed, state how the space can be reduced. **[1 marks]**

CSE221 Assignment 4[WBH]

Summer 2026

Section 14

Instructions:

- Assignment must be handwritten
- Naming convention: **StudentID_A4**, example: 12101100_A4
- Submit scanned pdf in this google form: https://forms.gle/VzPMPioB4JkG2cNU9
- Deadline(no extensions): **12th September, 2026 (11:59Pm)**
- **Question 1 and 2g are optional**

Ensure assignment is submitted only using this google form, submission elsewhere or after the deadline will not be accepted. Must submit using your g-suite email.

Question 1[optional] **(10 marks)**

Place N queens on an N x N chessboard so that no two queens attack each other. For this question, use N = 8 and place one queen in each column.

(a) State the three conflict conditions that must be checked before placing a queen at row r and column c. Explain how rows and the two diagonal directions can be represented for fast safety checks. **[2 marks]**

(b) Write a recursive backtracking pseudocode that tries rows 1 through 8 in each column, places a safe queen, recurses to the next column, and removes the queen when that branch fails. **[4 marks]**

(c) Give one valid arrangement of eight queens as a list where the value at position c is the selected row for column c. Verify briefly that no two listed queens share a row or diagonal. **[2 marks]**

(d) Explain how pruning improves on generating every board arrangement. State a reasonable worst-case time bound and the auxiliary-space complexity of the backtracking solution. **[2 marks]**

Question 2 **(10 marks)**

A lab has one machine and receives the following half-open reservation requests [start, finish).

A	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**
[1,4)	[3,5)	[0,6)	[5,7)	[3,9)	[5,9)	[6,10)	[8,11)	[8,12)	[2,14)	[12,16)

(e) Apply the earliest-finish-time greedy algorithm. Show the sorted order, each accept/reject decision, and the maximum compatible set. **[4 marks]**

(f) Give a brief exchange-argument justification for why choosing the compatible interval that finishes earliest is optimal for the unweighted problem. **[2 marks]**

(g) (optional)

For the weighted requests P=[1,3), value 5; Q=[3,5), value 5; and R=[1,5), value 13, show that earliest-finish-time greedy is not optimal. Explain why weighted interval scheduling needs a different method, and write the dynamic-programming recurrence using p(j), the last request compatible with j. **[4 marks]**

Question 3 (10 marks)

A telemetry system observes the following symbol frequencies. When frequencies tie, place the alphabetically smaller symbol or subtree on the left. Treat left as 0 and right as 1.

A	B	C	D	E	F
32	25	20	12	7	4

(h) Simulate Huffman's greedy algorithm using a min-priority queue. Show every merge and draw the final Huffman tree. **[4 marks]**

(i) Write the codeword for each symbol and calculate the total number of bits needed to encode all 100 symbol occurrences. **[3 marks]**

(j) Encode the message ABACA. Compare its length with fixed-length coding for six symbols and state the number of bits saved or lost. Ignore the code-table overhead. **[2 marks]**

(k) Explain briefly how the greedy choice in Huffman coding leads to an optimal prefix-free code. **[1 marks]**

Question 4 (10 marks)

In the 15-puzzle, the blank may swap with an adjacent tile. Consider the following initial and goal states.

	Initial state					Goal state		
1	2	3	4		1	2	3	4
5	6	7	8		5	6	7	8
9	10	11	12		9	10	11	12
_	13	14	15		13	14	15	_

(aa) Describe a state representation, the legal moves, and the inversion-parity test used to reject an unsolvable 4x4 puzzle before searching. **[2 marks]**

(bb) Write recursive backtracking pseudocode that uses a visited-state set, a path list, a depth limit, pruning, and undoing of moves. The procedure should return a solution path if one is found. **[4 marks]**

(cc) Using move order Right, Up, Left, Down, trace the successful branch from the initial state through the first three recursive levels and write the move sequence that reaches the goal. **[2 marks]**

(dd) Explain why plain depth-first backtracking does not guarantee the shortest solution. Name a method that does guarantee the shortest number of moves and compare its memory use with DFS backtracking. **[2 marks]**

Question 5 (10 marks)

A rescue aircraft has a capacity of 10 weight units. The available supply bundles are listed below.

Bundle	Weight	Value	Value/weight
A	2	60	30
B	4	100	25
C	6	120	20
D	5	95	19

(ee) Explain the difference between fractional and 0-1 knapsack, including whether an item may be divided and the standard algorithmic approach for each. **[2 marks]**

(ff) Solve the fractional-knapsack version using the greedy value-to-weight ratio. Show the selected fractions and maximum value. **[3 marks]**

(gg) For the 0-1 version, write the recurrence and base cases, construct the DP table for capacities 0 through 10, and report the selected bundles and maximum value. **[4 marks]**

(hh) Use your two results to explain why the fractional greedy rule is not guaranteed to solve 0-1 knapsack. **[1 marks]**

Question 6 (10 marks)

Let X = ABCBDAB and Y = BDCABA. For LCS(i, j), use the first i characters of X and the first j characters of Y.

(ii) Write the complete LCS recurrence relation, including base cases, and explain what each DP cell represents. **[3 marks]**

(jj) Construct the full dynamic-programming table, determine the LCS length, and backtrack to produce one valid LCS string. If several answers exist, any one is acceptable. **[4 marks]**

(kk) Explain overlapping subproblems and optimal substructure in the context of LCS. Refer to specific recursive subproblems to show how both properties arise. **[2 marks]**

(ll) State the DP time and space complexity. If only the LCS length is needed, state how the space can be reduced. **[1 marks]**